AAE4011: Artificial Intelligence in Unmanned Autonomous Systems

Group Project

Topic

AI-Powered Semantic Navigation for Indoor Mobile Robots

Group 10

Prithula Prashun Aishy (22098865D)

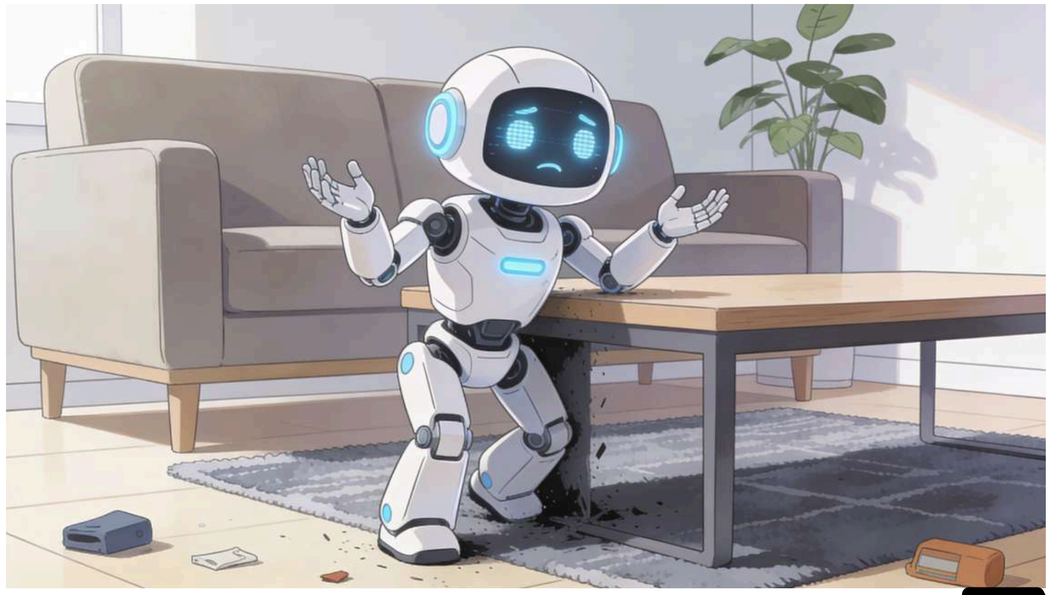
WU Chun Him (25016333D)

Table of Contents

1. Problem Statement
2. Project Overview
3. Technical Approach
4. Dataset and Class Mapping
5. Training Process
6. Cost Map Generation
7. PRM Path Planning
8. Results
9. Project Structure
10. How to Use the Project
11. Challenges and Solution
12. Future Improvements
13. Applications
14. Conclusion

Problem Statement

- Indoor mobile robots need to-
  - -understand scene context
  - -identify hazardous areas
  - -interpret visual scenes
  - -generate semantically safe path planning



This project solves this using **semantic segmentation** - teaching a neural network to classify every single pixel in an image into one of four navigation-relevant classes

Project Overview

Image
↓
Segmentation
↓
Cost Map
↓
Path planning

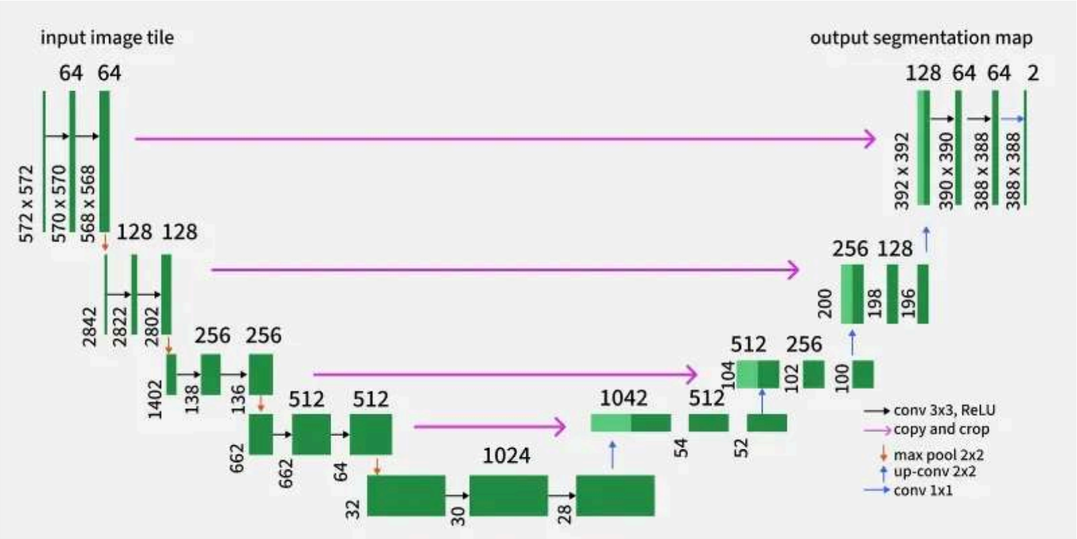
The pipeline has four main stages-

1. **Perception** - A CNN performs pixel-wise scene classification
2. **Semantic Mapping** - Convert classes to traversal costs
3. **Path Planning** - A* algorithm finds optimal paths
4. **Safety Margin** - Maintain distance from obstacles

Class 0: Floor - Safe to walk on
Class 1: Wall/Obstacle - Blocked
Class 2: Door - Passable transition
Class 3: No-go zones - Stairs, fragile areas (restricted)

Technical Approach

Transfer Learning with a **U-Net** architecture using a **ResNet50** encoder



U-Net-

- has an encoder-decoder structure
- skip connections preserve spatial information
- produces high-resolution output

ResNet50 is pre-trained on ImageNet with 1.4 million images hence understands-

- Edges, corners, and textures
- Basic shapes and patterns
- Common objects

Dataset and Class Mapping

ADE20K dataset ---> a large-scale dataset with 20,210 training images and 2,000 validation images covering diverse indoor scenes.

150 different classes

→

Floor (Class 0) - includes floor, carpet, rug, sidewalk, path etc.

Wall/Obstacle (Class 1) - includes wall, furniture, cabinets, tables etc.

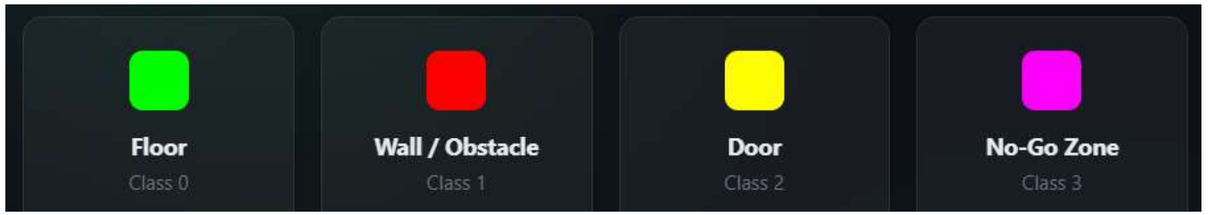
Door (Class 2) - includes door, doorway, screen door etc.

No-go (Class 3) - includes stairs, water, mountains, fireplace etc.

Dataset and Class Mapping

What the Mapper Produces?

- Output: a Python dictionary saved as a .pkl file mapping each ADE20K ID (1–150) → navigation class (0–3)
- This mapping is loaded once and applied to every training mask before the model sees it
- Training masks originally have values 1–150 -----> after remapping they only contain 0, 1, 2, or 3
- This pre-processing step ensures the model never sees the original 150-class labels



Data Preparation

How PyTorch Loads Data

- Created a custom ***IndoorNavigationDataset*** class by inheriting from PyTorch Dataset
- The dataset loads matched image-mask pairs from ADE20K training and validation folders
- Each sample goes through the following steps:
  - -Load RGB image using OpenCV
  - -Load grayscale annotation mask
  - -Resize both to 256 × 256
  - -Remap ADE20K labels into 4 navigation classes
  - -Apply transforms and convert to tensors
- Mask resizing uses nearest-neighbor interpolation so class labels are not corrupted
- If a corrupted image or mask is found, the code skips it and loads the next valid sample

Data Augmentation

Why it Matters?

- What we applied → random horizontal flip, rotation (±15°), brightness/contrast jitter

- A robot encounters rooms from different angles, under different lighting. Augmentation synthetically teaches the model to be invariant to these variations without collecting new data.

- Augmentation effectively multiplies the size of our training set and prevents the model from memorising specific image orientations.

Training Process

- Training images and annotation masks loaded from ADE20K
- Masks are remapped from ADE20K labels to 4 navigation classes
- Data augmentation includes resizing, flips, rotation, brightness, and contrast changes.
- Model is trained using weighted cross-entropy to handle class imbalance
- Validation loss is monitored and the best model is saved

Training Configuration

- Batch size: 16
- Learning rate: 1e-4
- 40 epochs
- Optimizer: AdamW with weight decay

The model achieved a **validation loss of 0.3663.**

Training Process

Learning Rate scheduling

- ***ReduceLROnPlateau*** monitors validation mIoU after every epoch
- If mIoU doesn't improve for 4 consecutive epochs (patience=4), the learning rate is cut in half (factor=0.5)
- Uses mode='max' because higher mIoU = better (contrast with loss where lower = better)
- This prevents overshooting the optimal solution as the model approaches convergence

Training Process

Evaluating the Model - mIOU

- mIoU = Mean Intersection over Union
- For each class: IoU = (Predicted ∩ Ground Truth) / (Predicted ∪ Ground Truth)
- mIoU = average of all 4 class IoU scores
- Perfect prediction = 1.0 | Random prediction ≈ 0.25 for 4 classes
- Our validation loss of 0.3663 reflects this metric being monitored every epoch by ReduceLROnPlateau
- Door is the lowest-IoU class because it is likely because it appears less frequently and varies greatly in appearance

Cost Map Generation

- A cost map is a 2D grid where every pixel has a numerical value representing how "expensive" it is for a robot to travel through that location

Segmentation → what each pixel is
Cost Map → wheather should go there

Class	Colour	Cost	Meaning
Floor		1.0	Safe
Wall		inf	Blocked
Door		0.8	Preferred
No-go		inf	Blocked

Cost Map Generation

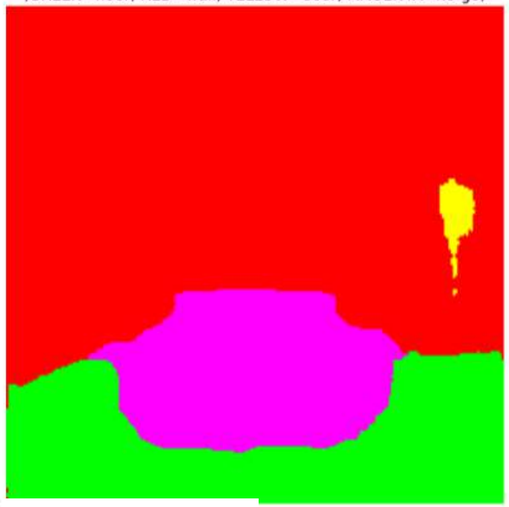
The process_image() method orchestrates the complete pipeline:

1. Convert input BGR image → RGB (for display and internal processing)
2. Run predict() → segmentation mask (class IDs per pixel)
3. Run get_colored_segmentation() → colour-coded visual for human inspection
4. Run get_cost_map() → numerical cost grid for the path planner
5. Run get_cost_map_visualization() → colour-coded cost map for debugging
6. Save outputs:
   - _cost_map.npy — raw float64 array (preserves inf values correctly)
   - _colored_segmentation.png — visual mask
   - _cost_viz.png — cost map image

Cost Map Generation

Original Image

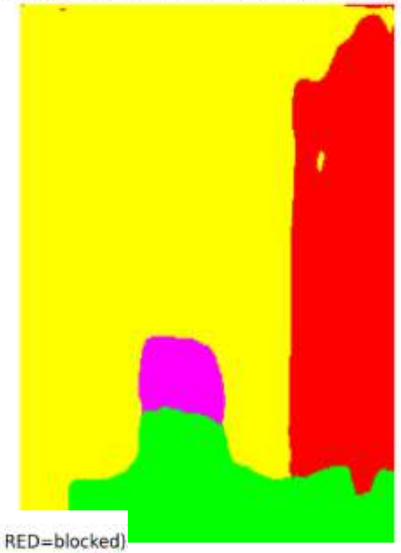
Segmentation
(GREEN=floor, RED=wall, YELLOW=door, MAGENTA=no-go)



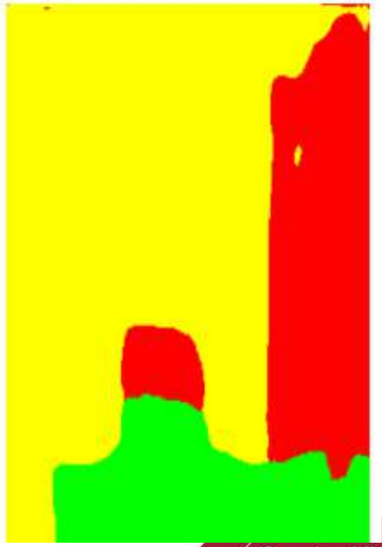
Cost Map
(GREEN=good, YELLOW=door, RED=blocked)

Original Image

Segmentation
(GREEN=floor, RED=wall, YELLOW=door, MAGENTA=no-go)



Cost Map
(GREEN=floor, YELLOW=door, RED=blocked)



Path Planning

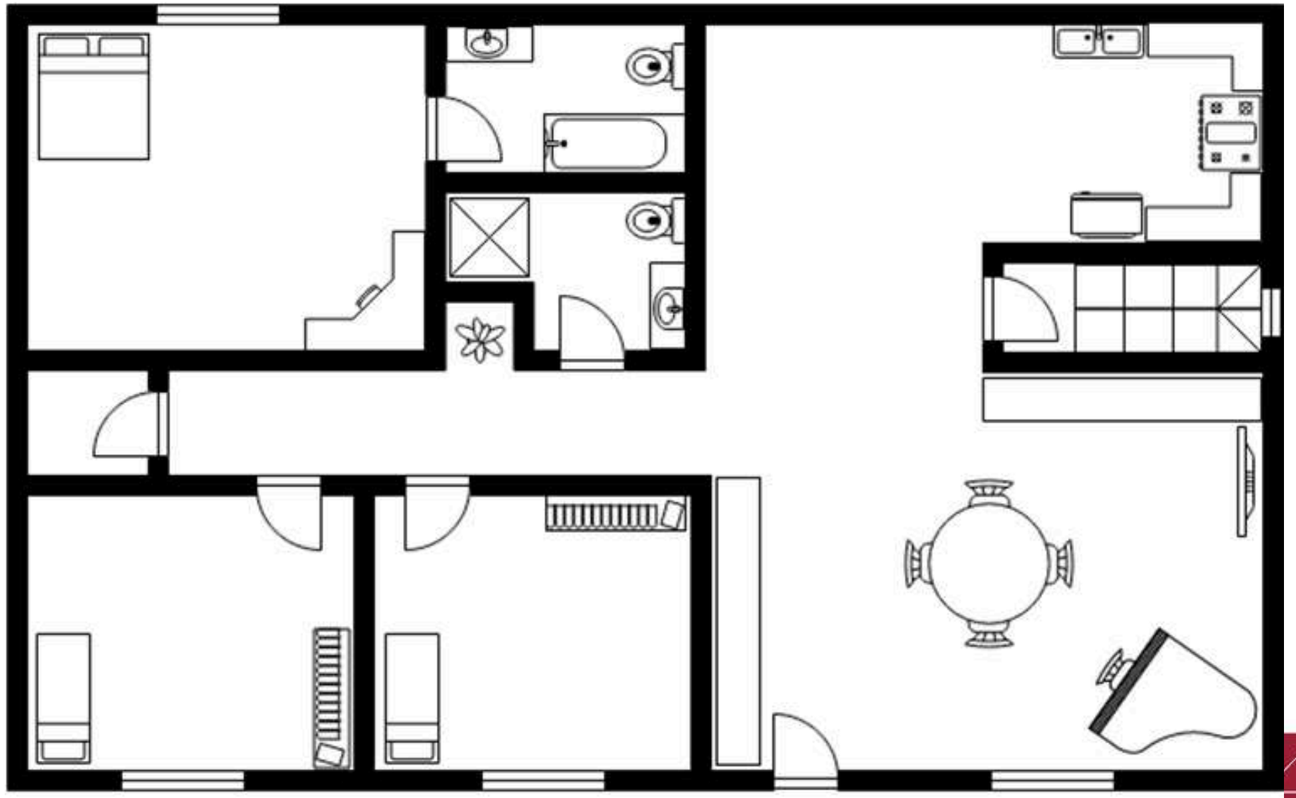
Probabilistic Roadmap (PRM)

- used in robotics and computer graphics to find suitable paths in complex environments
- marking the node in free space
- connected to nearest node to form a graph or roadmap.
- graph between nodes represents the possible paths that a robot can take and avoid obstacles.

Inflating obstacles

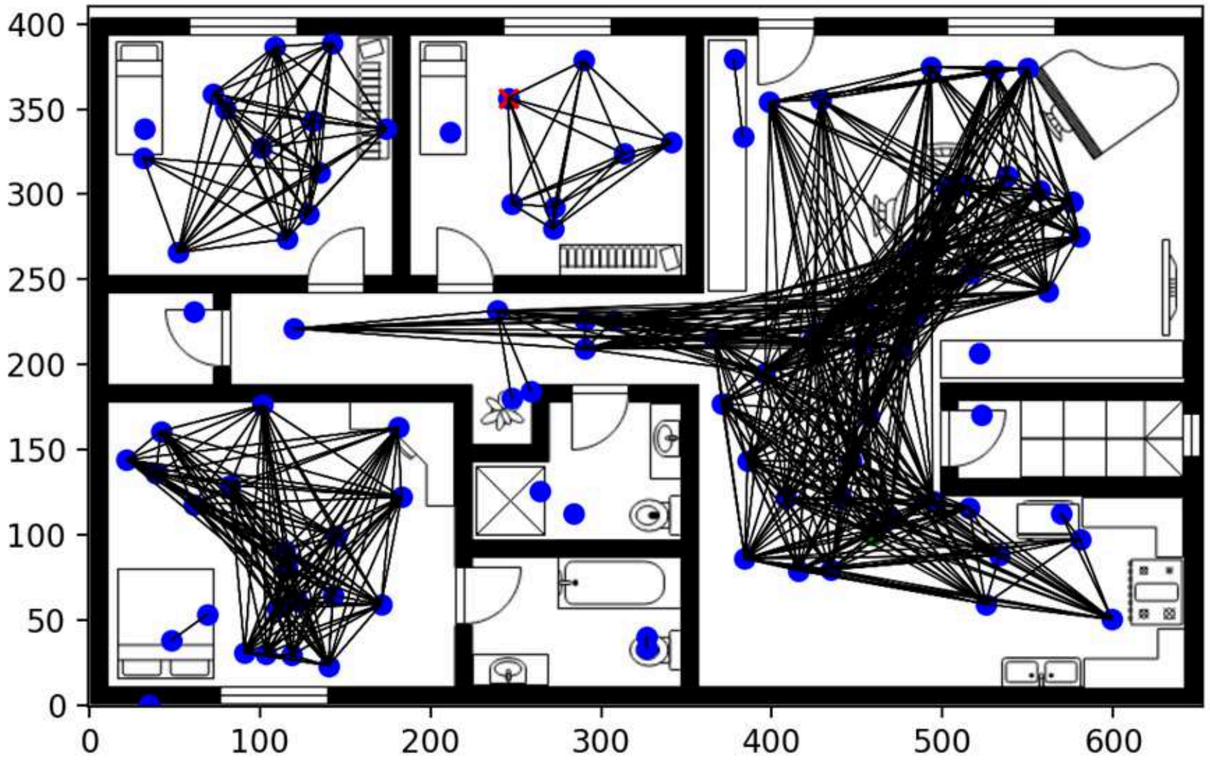
To avoid the robot getting too close to the obstacles

- Expanding the boundaries of each obstacle by a certain number of pixels
- Having a safety distance between robot and obstacle



Node's Connections

- Node connection is a important step
- The linked node will form a roadmap



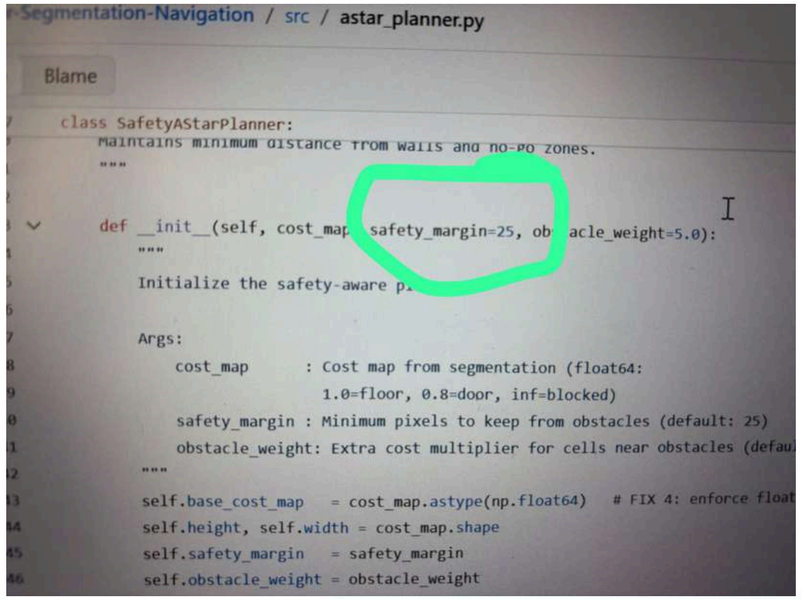
Bresenham's Line Algorithm

- Computes the next node from the current one using a small set of integer additions or subtractions and do comparisons
- Form a close approximation line between nodes

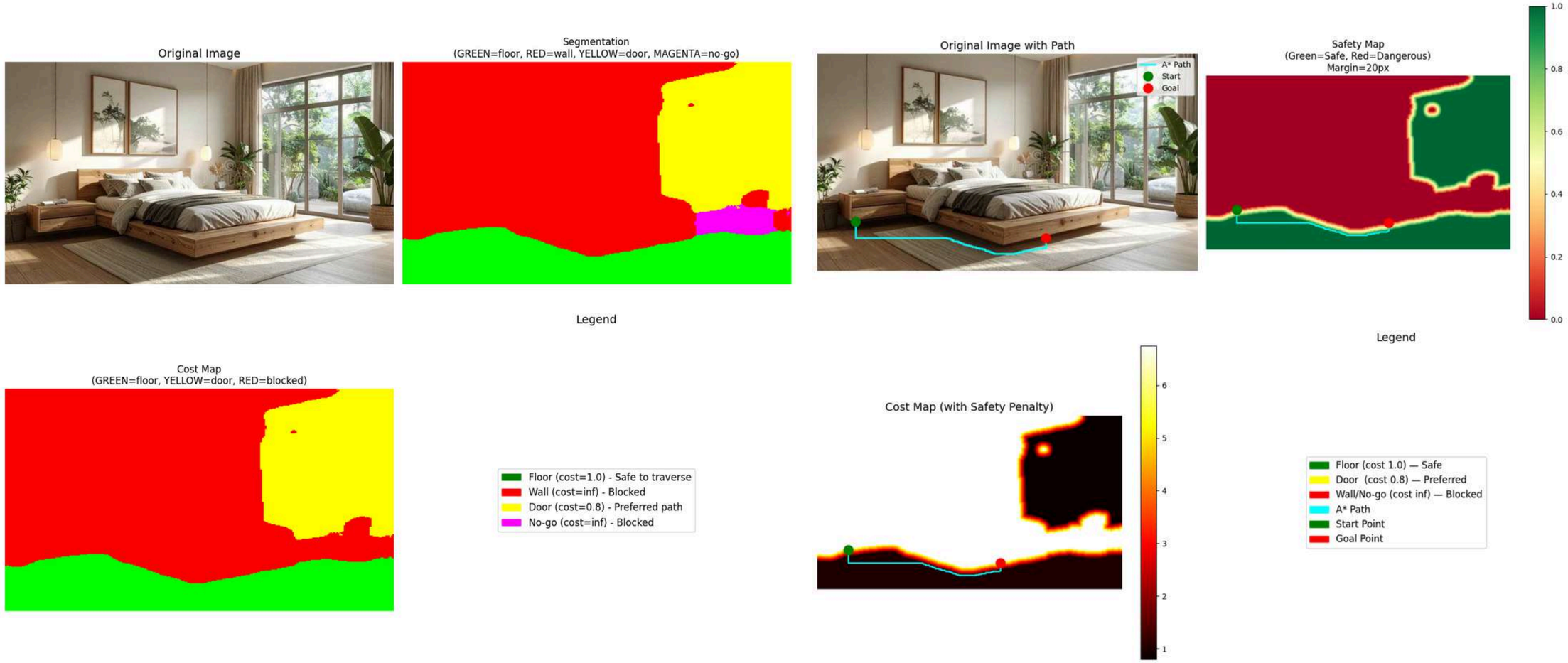
```
class SafetyAStarPlanner:
    def get_neighbors(self, node):
    # ----------------------------------------------------------------
    def find_valid_point(self, point, search_radius=100):
        """
        Find nearest traversable point if the given point is blocked.
        Uses distance_transform_edt for fast nearest-neighbour lookup (FIX 3).

        Args:
            point        : (y, x) tuple
            search_radius: pixel radius to search

        Returns:
            Valid (y, x) point, or None if not found
        """
        y, x = point

        # Already valid?
        if 0 <= y < self.height and 0 <= x < self.width:
            if self.traversable_mask[y, x]:
                return (y, x)

        # FIX 3: vectorized nearest traversable pixel via distance transform
```

A* Path Planning with Safety Margin

- Uses A* to find the lowest-cost path from start to goal
- Input is the cost map generated from semantic segmentation
- Blocked cells are detected using infinite cost
- A distance transform computes how far each pixel is from the nearest obstacle
- A safety penalty is added to cells near obstacles
- Final path prefers:
  - -traversable floor
  - -door regions
  - -open space far from walls



Result


Original Image
Segmentation
(GREEN=floor, RED=wall, YELLOW=door, MAGENTA=no-go)
Original Image with Path
A* Path
Start
Goal
Safety Map
(Green=Safe, Red=Dangerous)
Margin=20px
1.0
0.8
0.6
0.4
0.2
0.0
Legend
Cost Map
(GREEN=floor, YELLOW=door, RED=blocked)
Floor (cost=1.0) - Safe to traverse
Wall (cost=inf) - Blocked
Door (cost=0.8) - Preferred path
No-go (cost=inf) - Blocked
Cost Map (with Safety Penalty)
6
5
4
3
2
1
Legend
Floor (cost 1.0) — Safe
Door (cost 0.8) — Preferred
Wall/No-go (cost inf) — Blocked
A* Path
Start Point
Goal Point

Project Structure

```
Indoor-Segmentation-Navigation/
├── src/
│   ├── astar_planner.py
│   ├── class_mapper.py
│   ├── config.py
│   ├── cost_map_generator.py
│   ├── data_prep.py
│   ├── model.py
│   └── train.py
├── models/
│   └── .gitkeep
├── requirements.txt
└── README.md
```

- best_model.pth in drive--> Link to download provided
- The pipeline will automatically-

  - clone the repo
  - download the model from Google Drive
  - process uploaded image

Project available on: https://github.com/aishy22/Indoor-Segmentation-Navigation.git

How to Use the Project

Upload the Image---> Segmentation and Cost Map---->Path Planning

- Floor (cost=1.0) - Safe to traverse
- Wall (cost=inf) - Blocked
- Door (cost=0.8) - Preferred path
- No-go (cost=inf) - Blocked

- Floor (cost 1.0) - Safe
- Door (cost 0.8) - Preferred
- Wall/No-go (cost inf) - Blocked
- A* Path
- Start Point
- Goal Point

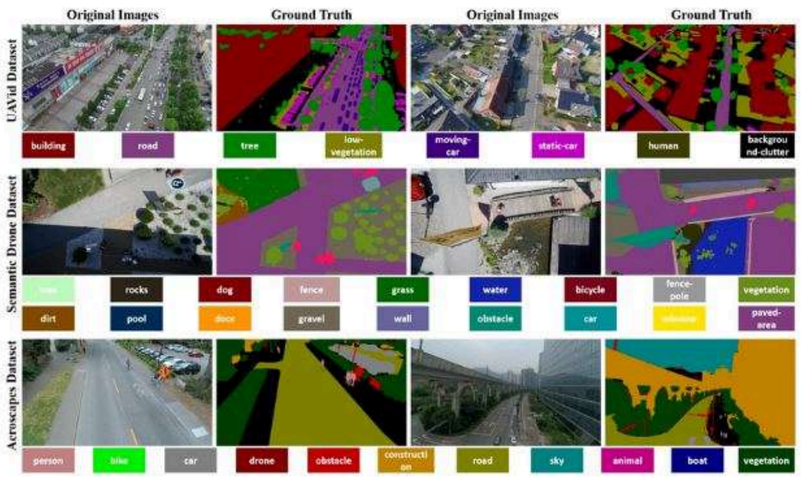
Challenges and Solution

Challenge	Solution
1. Large Dataset	Used Google Colab with GPU
2. Path Hugging Walls	Implemented safety margin in PRM
3. Overfitting	Added dropout, weight decay, early stopping

Future Improvements

- **Real-time video processing**

Extend from single images to live video streams by optimizing inference speed and implementing frame-to-frame consistency for smoother robot navigation.



- **Dynamic obstacle avoidance**

Incorporate real-time object detection to handle moving obstacles like people and pets, allowing the robot to dynamically re-plan paths.

- **Better door detection**

Collect more door examples to improve door detection accuracy, currently the weakest class due to limited training samples.

Applications

- Home robots - Roomba-style vacuums that understand room layouts
- Hospital Robots - Deliver medicine without bumping into equipment
- Warehouse robots - Navigate between shelves safely
- Assistive robots - Help visually impaired people navigate indoors

Conclusion

- Understands scenes semantically
- Generates cost maps
- Plans safe paths
- Processes images in 1-2 second

Thank You!
Q & A